

Table of Contents

Unit 1: Operating Systems — Structure and Services.....	6
A Complete Interactive Textbook Chapter for 10th Class Computer Science.....	6
Student Learning Outcomes.....	6
Introduction.....	7
1.1 Introduction to the Operating System (OS)	7
1.1.1 Operating System (OS) as the Central System Controller	7
Story Mode: The Birth of the First Operating System	7
What is an Operating System?	8
Pause & Think — 1.1.1	9
Quick Recap	9
1.1.2 Role in User–Hardware Interaction	9
Story Mode: Lost in Translation.....	9
The OS as a Translator	10
Practical Walkthrough: Watch Your Keyboard Signal in Real Time.....	10
Pause & Think — 1.1.2	11
Quick Recap	11
1.1.3 Responsibilities in Multi-User Environments	11
Story Mode: The Computer Lab Problem	11
What the OS Does in Multi-User Environments	12
Pause & Think — 1.1.3	12
Quick Recap	12
1.1.4 Creating and Managing User Accounts	12
How the OS Manages User Accounts	13
Practical Walkthrough: Creating a New User Account on Windows.....	13
Pause & Think — 1.1.4	14
Quick Recap	14
1.2 Architecture of an Operating System.....	14
1.2.1 Kernel vs. Shell.....	14
Story Mode: The Birth of Unix — Engine and Dashboard.....	14
Understanding the Kernel.....	15
Understanding the Shell	15
The Layered View: User → Shell → Kernel → Hardware	16
Practical Walkthrough: Exploring the Shell (Command-Line)	16
Grab a Partner — 1.2.1 (Role-Play).....	17
Quick Recap	17

1.2.2 OS Layers and Modular Design	17
Story Mode: Building a Skyscraper, One Floor at a Time	17
Understanding OS Layers	18
Real-World Analogy: Your School System	19
Pause & Think — 1.2.2	19
Quick Recap	19
1.2.3 System Libraries and Device Drivers	19
System Libraries: The Ready-Made Toolbox	19
Device Drivers: The Hardware Translators	20
Practical Walkthrough: Viewing Device Drivers on Your Computer	21
Pause & Think — 1.2.3	22
Quick Recap	22
1.3 Process Management in Operating System (OS)	22
1.3.1 Process Life Cycle	22
Story Mode: Margaret Hamilton and the Moon Landing	22
What is a Process?	23
The Process Life Cycle	23
The Three Main Stages: Creation, Execution, Termination	24
Real Example: Opening Google Chrome	25
Pause & Think — 1.3.1	25
Quick Recap	26
1.3.2 Multitasking and Concurrency	26
Story Mode: The Master Chef	26
Multitasking	26
Concurrency	27
Multitasking vs. Concurrency — A Simple Comparison	27
Pause & Think — 1.3.2	27
Quick Recap	28
1.3.3 Process Scheduling Concepts	28
The Problem: One CPU, Many Waiting Processes	28
First-Come, First-Served (FCFS) Scheduling	28
Numerical Example: FCFS in Action	28
CPU Timeline Diagram	29
Waiting Time Calculation	29
Advantages and Disadvantages of FCFS	30
Practical Walkthrough: Watch Real Processes on Your Computer	30
Pause & Think — 1.3.3	31
Quick Recap	31
1.4 Memory	31

1.4.1 Primary Memory (RAM)	31
Story Mode: The Desk and the Filing Cabinet	31
What is RAM?	32
Why RAM Matters for Performance	32
Practical Walkthrough: Check Your Computer's RAM	33
Pause & Think — 1.4.1	33
Quick Recap	34
1.4.2 Virtual Memory	34
Story Mode: The Manchester Atlas and the Magic Trick	34
What is Virtual Memory?	34
How Virtual Memory Works	35
The Trade-Off: Virtual Memory is Slower	35
Practical Walkthrough: Check Virtual Memory Settings on Windows	36
Pause & Think — 1.4.2	36
Quick Recap	37
1.5 Processes vs. Threads	37
Story Mode: The Restaurant Kitchen	37
Process vs. Thread — The Core Difference	38
The Web Browser Example	38
Pause & Think — 1.5	39
Quick Recap	39
1.5.1 Multithreading	39
What is Multithreading?	39
1.5.2 Benefits of Multithreading	40
1. Enhanced Performance	40
2. Improved Responsiveness	40
3. Support for Concurrent Operations	40
4. Efficient Use of Resources	41
Grab a Partner — 1.5.2	41
Quick Recap	41
1.6 System Calls	42
Story Mode: The Catastrophic Early Days of Personal Computing	42
What is a System Call?	42
1.6.1 Types of System Calls	43
1. open()	43
2. read()	43
3. write()	43
4. fork()	44
System Call Summary Table	44

Pause & Think — 1.6.1	44
Quick Recap	45
1.7 File System Structure and Management	45
Story Mode: The Library Before Card Catalogues	45
1.7.1 Files	45
1.7.2 Folders	46
1.7.3 Metadata	47
Practical Walkthrough: Viewing File Metadata	47
Pause & Think — 1.7.3	48
1.7.4 File Systems	48
Pause & Think — 1.7.4	49
Quick Recap	49
1.8 Types of Operating Systems	50
Overview	50
1.8.1 Real-Time Operating System (RTOS)	50
Story Mode: The Pacemaker and the Deadline	50
What is an RTOS?	50
Pause & Think — 1.8.1	51
Quick Recap	51
1.8.2 Embedded Operating System (EOS)	52
Story Mode: The Computer Inside Your Microwave	52
What is an EOS?	52
Pause & Think — 1.8.2	53
Quick Recap	53
1.8.3 Network Operating System (NOS)	53
Story Mode: The School Computer Lab	53
What is a NOS?	54
NOS vs. General-Purpose OS	55
Pause & Think — 1.8.3	55
Quick Recap	55
1.8.4 Mobile Operating System	55
Story Mode: From Symbian to Smartphones	55
What is a Mobile OS?	56
Major Mobile Operating Systems	56
Mobile OS vs. Desktop OS	57
Practical Walkthrough: Explore Your Mobile OS	57
Pause & Think — 1.8.4	57
Quick Recap	58

Chapter Summary	58
Review Questions.....	59
Section A: Recall (Answer in one or two sentences)	59
Section B: Understanding (Answer in 4–6 sentences)	59
Section C: Application and Analysis	59
Glossary of Key Terms	61

Unit 1: Operating Systems — Structure and Services

A Complete Interactive Textbook Chapter for 10th Class Computer Science

To the Student: You use an operating system every single day. Every time you tap your phone, open a game, or save a school assignment, an operating system is silently doing hundreds of things on your behalf. By the end of this chapter, you will understand *exactly* what is happening — and you will never look at your computer or phone the same way again. Let us begin.

Student Learning Outcomes

By the end of this chapter, you will be able to:

- Define an operating system and explain its role as an interface between the user and hardware
 - Identify the responsibilities of the OS in managing multiple user accounts
 - Differentiate between the kernel and shell, including their specific functions
 - Explain OS layers and how each layer interacts with the others
 - Identify the role of system libraries and device drivers in OS functionality
 - Explain the stages in the process lifecycle (creation, execution, termination) and apply First-Come, First-Served (FCFS) scheduling
 - Describe multitasking and concurrency, giving one real-world analogy
 - Distinguish between primary memory (RAM) and virtual memory
 - Define a process and a thread, and explain how threads share resources within a process
 - Give examples of how multithreading improves performance in applications
 - Define a system call and explain its purpose and types
 - Define file system, files, folders, and metadata, and explain their roles in organising data
 - Explain how all these components work together to process a user request from device to server and back
-

Introduction

Imagine it is the early 1950s. You work at a university computer centre. A massive machine — the size of a large room — sits humming in front of you. You need to run a program. You punch your instructions onto paper cards, one hole per instruction, and hand the stack to an operator. The operator manually loads your cards into the machine. The computer runs your program, prints the result, and then stops completely — waiting in silence — until a human physically loads the next program.

One program. One user. One task at a time.

That was computing before operating systems existed.

Today, your phone runs dozens of applications at the same time, plays music, downloads messages, tracks your location, and checks for software updates — all without you doing anything. A small, invisible piece of software makes all of this possible.

That software is called an **operating system**.

In this chapter, we will go inside the operating system. We will explore its architecture, understand how it manages programs, memory, files, and users, and see exactly how it keeps your computer running smoothly and safely. This chapter covers:

- How the OS controls all hardware and software on your computer
- How it manages running programs (called processes)
- How it organises memory so programs do not interfere with each other
- How files and folders are stored and retrieved
- The different types of operating systems in use today

Ready? Let us start.

1.1 Introduction to the Operating System (OS)

1.1.1 Operating System (OS) as the Central System Controller

Story Mode: The Birth of the First Operating System

It is 1956. General Motors — the car company — has a serious problem. Its IBM 704 computer is powerful but wasteful. The machine can process data at incredible speed, but it spends most of its time sitting idle, waiting for human operators to manually swap programs in and out. Each program hand-off takes minutes. The computer itself takes only seconds to finish a task.

A team of engineers asked: *What if we wrote a program to automatically manage and load other programs?*

In 1956, they built **GM-NAA I/O** — widely considered the world's first operating system. Instead of humans managing tasks, a program did it automatically. The computer's idle time dropped dramatically.

The central controller was born.

What is an Operating System?

An **operating system (OS)** is a special program that:

1. Controls all the hardware in your computer (CPU, memory, screen, keyboard)
2. Runs all other software (games, browsers, word processors)
3. Provides an interface — a way for *you* to communicate with the machine

Think of the OS as a **school principal**. The principal does not teach classes personally. Instead, the principal manages classrooms, schedules bells, assigns teachers to rooms, distributes supplies, and makes sure the whole school runs in order. If two teachers need the same classroom at the same time, the principal resolves the conflict.

The operating system does exactly this — but for your computer's resources.

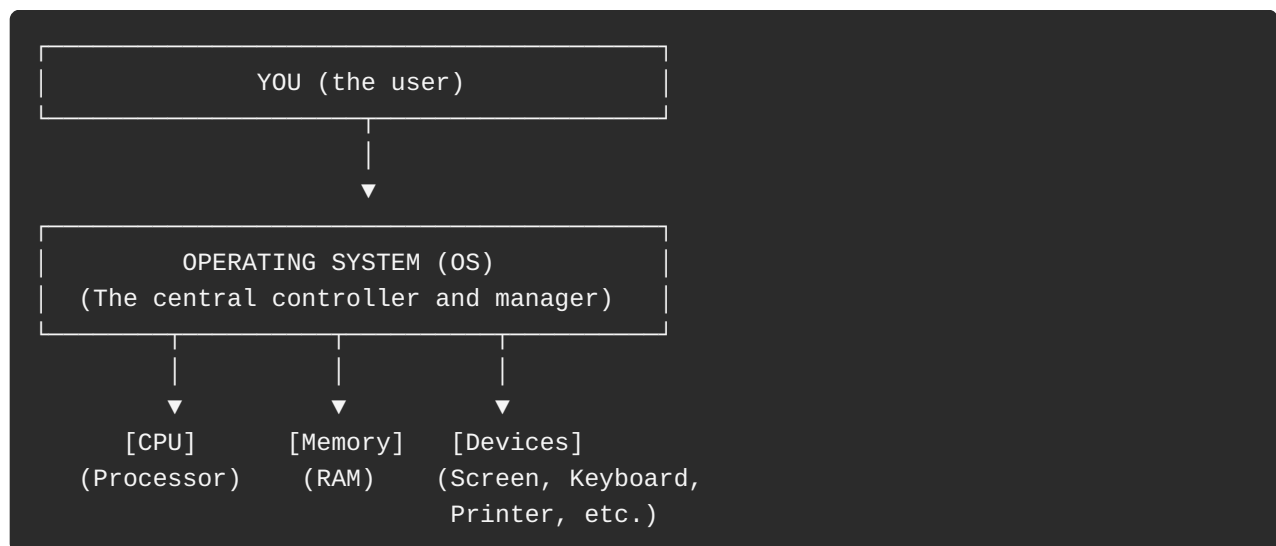


Figure 1.1: The OS sits between you and all the hardware. Nothing reaches the hardware without going through the OS.

Examples of operating systems you already use:

- **Windows 11** — on most school or home computers
- **macOS** — on Apple MacBook computers

- **Linux** — on servers, scientific computers, and many developer machines
 - **Android** — on most smartphones
 - **iOS** — on iPhones and iPads
-

Pause & Think — 1.1.1

You are using a computer with no operating system — just the raw hardware. You want to type a letter and save it.

Think about this: What would you have to do manually, in electrical signals, to make the keyboard work? To display a letter on screen? To save a file to the hard drive?

Discuss with a classmate why this would be nearly impossible for an ordinary person.

Quick Recap

The operating system is the central controller of your computer. It manages all hardware resources and makes it possible for you to run programs without understanding how the hardware works. Think of it as the school principal of your computer system.

1.1.2 Role in User–Hardware Interaction

Story Mode: Lost in Translation

Imagine you speak only Urdu and you need to give instructions to a construction worker who speaks only Mandarin Chinese. Without a translator, nothing gets built. You would stand there, pointing and hoping.

Computers have exactly this problem. You speak human language — Urdu, English, or another tongue. Your CPU speaks only **binary** — a language of zeros and ones, representing electrical signals that are either OFF (0) or ON (1).

When you press the letter **A** on a keyboard, the OS instantly translates your action into binary code — **01000001** — and sends it racing through circuits at nearly the speed of light to appear on your screen.

You do not see any of this. The translation happens invisibly, in microseconds.

The OS as a Translator

The operating system acts as a **translator and bridge** between the human user and the computer hardware.

When you do something — click, type, swipe, tap — here is what actually happens:

```
graph TD; A[You press the 'A' key] --> B[Keyboard sends an electrical signal]; B --> C["OS catches the signal and looks up:  
\"Which key? A. What is 'A' in binary? 01000001\""]; C --> D[OS sends 01000001 to the screen driver]; D --> E[Screen displays: A];
```

This entire chain happens in **less than one millisecond**. It feels instant because it practically is.

Why is this important?

Without the OS doing this translation:

- Every app developer would have to learn the electrical language of every different keyboard, screen, and processor on the planet
- You would need to understand binary code to use a computer
- Switching from one type of hardware to another would break every program ever written

The OS creates a **universal layer** — programs talk to the OS, and the OS talks to the hardware. This is why the same app can run on many different laptops, even though those laptops may have different processors and screens inside.

Practical Walkthrough: Watch Your Keyboard Signal in Real Time

On Windows:

1. Press the **Windows key** on your keyboard.
2. Type **Notepad** and press **Enter**.
3. The Notepad application opens.
4. Now press any key — say, the letter **B**.
5. The letter **B** appears on screen.

What just happened: You pressed a key → the OS detected an electrical signal from the keyboard → translated it to the character 'B' → passed it to Notepad → Notepad displayed it on screen. You witnessed all five steps of user-hardware translation in under a second.

Pause & Think — 1.1.2

If an application (like a game) could directly control your screen, keyboard, and speakers *without* asking the OS for permission:

- Could a badly written game accidentally break your keyboard settings?
- Could a malicious app secretly record every key you press?
- Could two apps fighting over the same speaker cause your audio to crash?

Write down **two security risks** that would exist if apps bypassed the OS and had direct hardware access.

Quick Recap

The OS acts as a translator between human language and binary hardware language. Every keystroke, tap, or click passes through the OS, which converts it into instructions the hardware can understand. This keeps computing simple for users and safe for the system.

1.1.3 Responsibilities in Multi-User Environments

Story Mode: The Computer Lab Problem

Picture a computer laboratory with 30 students, each working on a different assignment. All 30 students are logged into the same server. One student is writing an essay. Another is running a Python program. A third is watching a tutorial video.

Now imagine if there were *no* rules. Any student could open any other student's files. A program run by one student could slow down or crash the programs of all other students. The student running the video could use all the available processing power, freezing everyone else's screens.

Without management, a shared computer system becomes chaos.

The operating system is the manager that prevents this chaos.

What the OS Does in Multi-User Environments

In schools, offices, banks, and online platforms, many people use the same computing system. The OS ensures:

Responsibility	What It Means
User Isolation	Each user's files and data stay private. You cannot see another user's documents.
Resource Sharing	The CPU, memory, and printer are shared fairly. No single user can monopolise them.
Security	Each user must authenticate (prove who they are) with a username and password before gaining access.
Accountability	The OS keeps logs of who logged in, when, and what they did.
Protection	If one user's program crashes, it does not crash the programs of other users.

Pause & Think — 1.1.3

You are the OS in your school's computer lab. Three things happen simultaneously:

1. Student A is running a video editing program that is using 80% of the CPU
2. Student B is trying to print an important assignment
3. Student C just logged in and wants to open a web browser

How would you, as the OS, manage these three competing needs fairly?

Who gets priority? Who waits? Why?

Discuss your answer with a partner and write down your reasoning.

Quick Recap

In environments where many users share one computer or server, the OS creates separate, protected spaces for each user, shares resources fairly, and prevents any one user from disrupting others. This makes computing in schools, offices, and online services possible.

1.1.4 Creating and Managing User Accounts

How the OS Manages User Accounts

An operating system allows multiple separate **user accounts** on one machine. Each account has:

- Its own **username** (a unique name that identifies the user)
- Its own **password** (a secret code to prevent others from logging in)
- Its own **files and desktop** (separate storage space)
- Its own **settings** (wallpaper, language, preferences)
- A specific **account type** (which determines what the user is allowed to do)

Account Types:

Type	What They Can Do
Administrator	Full control — can install software, change system settings, create or delete other accounts
Standard User	Can use installed programs and change personal settings, but cannot install new software or change system-wide settings
Guest	Very limited access — can use the computer temporarily with no ability to save personal files

Practical Walkthrough: Creating a New User Account on Windows

1. Click the **Start** button (Windows logo at the bottom left).
2. Click **Settings** (the gear icon).
3. Click **Accounts**.
4. In the left panel, click **Family & other users**.
5. Under "Other users," click **Add account**.
6. Windows will ask: *Does this person have a Microsoft account?* Click **I don't have this person's sign-in information**.
7. Click **Add a user without a Microsoft account**.
8. Enter a **username** (e.g., "Student01") and a **password**.
9. Click **Next**.
10. Your new account appears in the list.

What just happened? The OS created a completely separate environment on the same machine. If you log in as Student01, you will see a blank desktop with no access to files from other accounts.

Note for teachers: You may demonstrate this step live. Students should not change settings on school computers without permission.

Pause & Think — 1.1.4

A school sets up a computer with three accounts: Teacher (Administrator), Student (Standard), and Library (Guest).

Consider these situations:

- The Student account tries to install a game. *What happens? Why?*
- A Guest user tries to save a file to the desktop. *Will it still be there after logout? Why?*
- The Teacher account accidentally installs a virus while downloading a program. *Could the virus affect the Student account's files?*

Think through each scenario and write a one-sentence answer for each.

Quick Recap

The OS manages user accounts by giving each person a separate, secure environment. Different account types — Administrator, Standard, and Guest — control what each user is allowed to do on the system. This keeps data private and the system safe.

1.2 Architecture of an Operating System

1.2.1 Kernel vs. Shell

Story Mode: The Birth of Unix — Engine and Dashboard

It is 1969. A small team at Bell Laboratories in the United States — including Ken Thompson and Dennis Ritchie — is building a new operating system called **Unix**.

They made a brilliant design decision: they separated the operating system into two distinct parts.

The **inner part** would directly control the computer hardware — the CPU, memory, storage. They called this the **kernel** (from the word for the seed at the centre of a nut).

The **outer part** would interact with the user — taking typed commands and passing them to the kernel. They called this the **shell** (the outer layer that protects and accesses the kernel).

This separation — kernel inside, shell outside — became the foundation of nearly every modern operating system, including Linux, macOS, Android, and iOS.

Understanding the Kernel

The **kernel** is the deepest, most powerful part of the operating system. It runs directly on top of the hardware.

What the kernel does:

- Controls the CPU — decides which program runs and for how long
- Manages memory — decides which program gets how much RAM
- Controls hardware devices — keyboards, screens, storage drives, network cards
- Handles security — prevents programs from accessing things they are not allowed to

Analogy: Think of a car. The **engine** is hidden under the hood. You cannot see it while driving. It does the actual work — burning fuel, turning wheels. The kernel is the engine of the operating system.

Understanding the Shell

The **shell** is the outer layer of the OS that the user directly interacts with.

What the shell does:

- Receives commands from the user (by clicking, typing, or tapping)
- Translates those commands into requests for the kernel
- Returns the kernel's results back to the user in a readable form

There are two types of shells:

Type	Description	Example
Graphical Shell	Visual interface with icons, windows, and menus	Windows Desktop, macOS Finder
Command-Line Shell	Text-based interface where you type commands	Windows Command Prompt, Linux Terminal

Analogy: In the same car, the **steering wheel, dashboard, and pedals** are the shell. These are the controls you interact with. You turn the wheel (input) and the car turns (output). You do not need to understand how the engine works to drive.

The Layered View: User → Shell → Kernel → Hardware

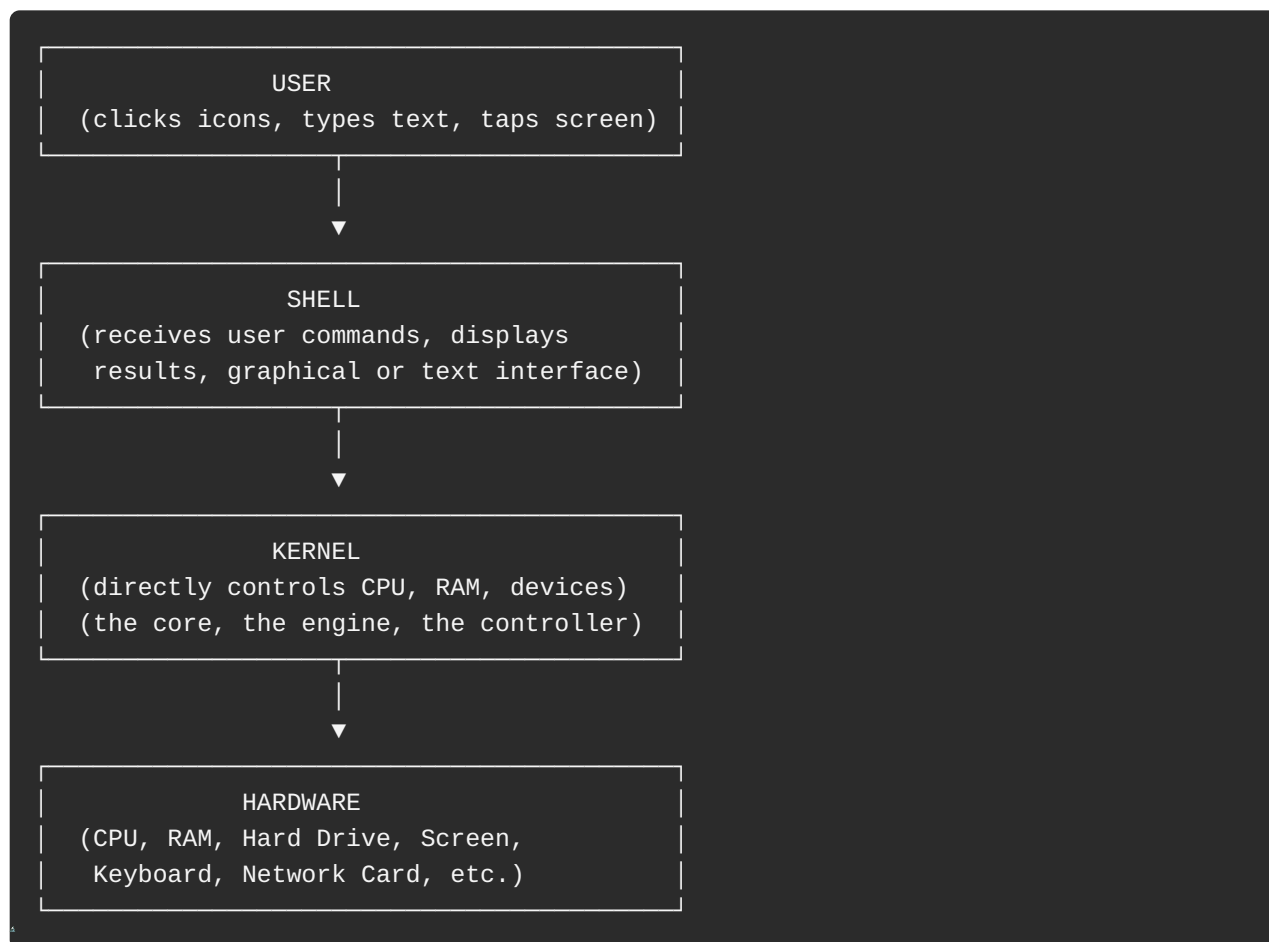


Figure 1.2: The four-layer view of how a user's action reaches the hardware.

Practical Walkthrough: Exploring the Shell (Command-Line)

Even if you usually use a graphical shell (the Windows desktop), you can also access the command-line shell to talk directly to the OS.

On Windows:

1. Press the **Windows key** and type `cmd`, then press **Enter**.
2. A black window opens — this is the **Command Prompt** (a command-line shell).
3. Type the following command and press **Enter**:

`dir`

4. The OS lists all the files and folders in your current location.
5. Now type:

`cd Desktop`

This command moves you to the Desktop folder.

6. Type `dir` again to see what is on your Desktop.

What just happened? You gave commands directly to the shell using text. The shell passed your commands to the kernel, which read the file system and returned the list. This is exactly what the graphical shell (your desktop icons) does — but you are now seeing the raw text version.

Grab a Partner — 1.2.1 (Role-Play)

Partner A plays the role of the **Shell**.

Partner B plays the role of the **Kernel**.

Role-play these three scenarios:

1. The user wants to open a photo.
 - Shell (A): Translate "open photo.jpg" into a hardware instruction.
 - Kernel (B): Describe what physical action happens (find the file on the drive, load it to memory, display it).
2. The user wants to print a document.
 - Shell (A): Receive the "Print" click and translate it.
 - Kernel (B): Describe how the kernel communicates with the printer driver.
3. The user wants to increase the speaker volume.
 - Shell (A): Translate the volume slider movement.
 - Kernel (B): Describe the hardware interaction with the audio chip.

After each scenario, swap roles.

Quick Recap

The **kernel** is the engine of the OS — it directly controls the hardware. The **shell** is the dashboard — it lets the user give commands. The user never touches the kernel directly; the shell handles that communication. Every OS, from Windows to Android, is built on this separation.

1.2.2 OS Layers and Modular Design

Story Mode: Building a Skyscraper, One Floor at a Time

A skyscraper is not built all at once. First, engineers lay the foundation and build the basement. Then the structural floors go up. Then the interior (plumbing, electricity). Then the exterior (windows, walls). Finally, the top floors that people actually use.

Each floor depends on the floor below it. If the foundation fails, the entire building falls. But if you want to redecorate an office on the 40th floor, you do not need to touch the foundation at all.

Operating systems are designed exactly like skyscrapers — in independent layers, each one building on the one below.

Understanding OS Layers

The OS is divided into **layers**. Each layer has a specific job, and each layer communicates only with the layers directly above and below it.

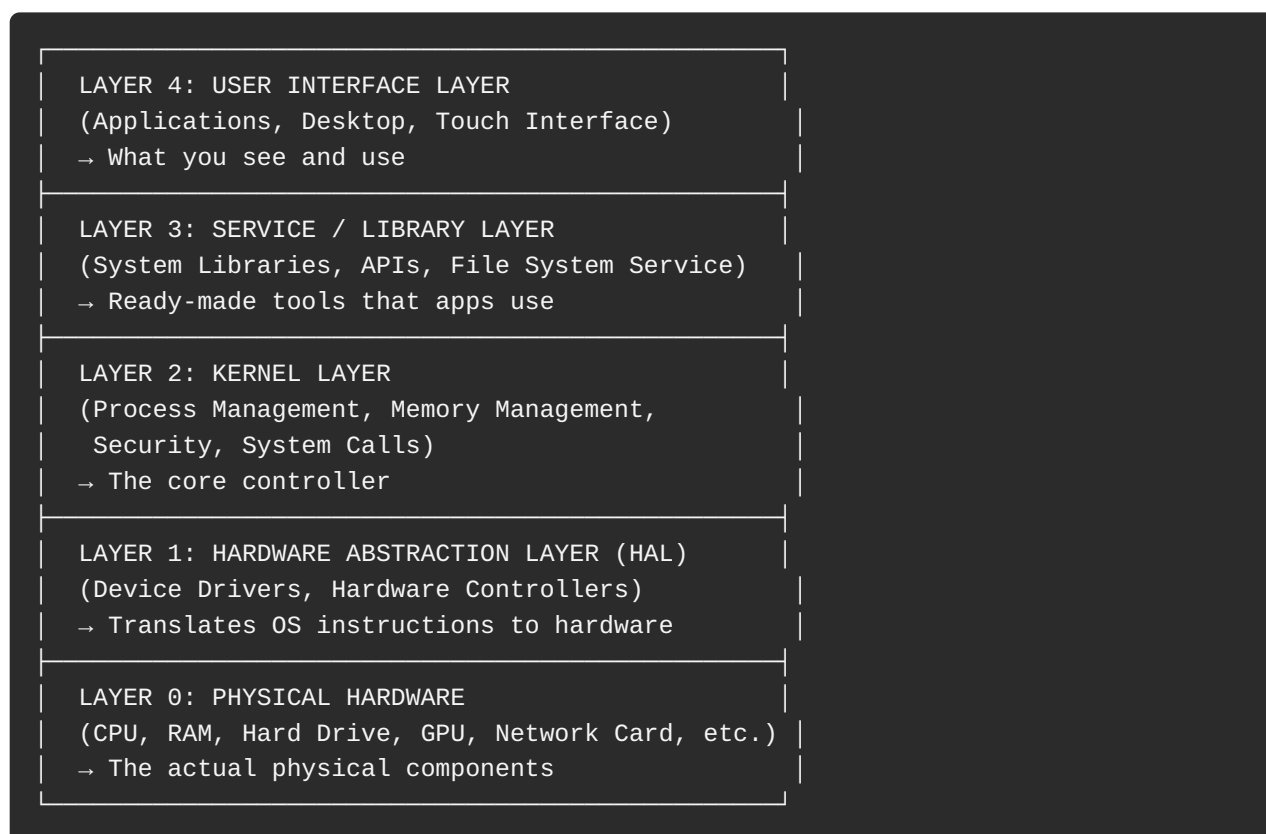


Figure 1.3: The layered architecture of an operating system. Each layer serves the layer above it.

Why is this layered design good?

- **Easier to fix:** If a device driver (Layer 1) has a problem, you only need to update that layer — not rewrite the entire OS.
 - **Easier to upgrade:** You can install a new graphics card (Layer 0) and only update its driver (Layer 1) without changing any applications (Layer 4).
 - **Safer:** Layers enforce boundaries. A user application (Layer 4) cannot directly access hardware (Layer 0) — it must request permission through the kernel (Layer 2). This prevents accidents and attacks.
-

Real-World Analogy: Your School System

School Role	OS Layer Equivalent
School building, electricity, desks	Physical Hardware
Guards, cleaners, maintenance staff	Hardware Abstraction Layer
Administration (schedules, rules, resource allocation)	Kernel Layer
Library resources, shared equipment	Service/Library Layer
Teachers and students doing actual work	User Interface & Applications

Each group serves the one above it. A student does not speak directly to the janitor to get a textbook — there is a system (library) for that. Similarly, an app does not speak directly to the CPU — there is a system (kernel) for that.

Pause & Think — 1.2.2

A student installs a new printer at home. The printer manufacturer provides a small program called a "driver" (we will study drivers in 1.2.3).

Which layer of the OS does this driver belong to?

If the driver has a bug, which other layers might be affected?

Would you need to reinstall your entire OS to fix a broken printer driver?

Write your answers and explain your reasoning.

Quick Recap

Operating systems are built in layers — like a skyscraper. Each layer has a specific job and communicates only with the layers next to it. This modular design makes the OS easier to update, safer to use, and more reliable when individual parts fail or change.

1.2.3 System Libraries and Device Drivers

System Libraries: The Ready-Made Toolbox

Imagine you want to bake a cake. You could grow wheat, mill it into flour, raise a cow for milk, and collect eggs from a chicken. Or you could go to a shop and buy everything pre-prepared.

System libraries are the pre-prepared ingredients of software development.

A **system library** is a collection of ready-made, tested code that programs can use to perform common tasks — without having to write those tasks from scratch.

Examples:

Task	System Library Does the Work
A photo app needs to open a JPEG file	It calls the OS image-reading library — no need to write image decoding code
A music player needs to output audio	It calls the OS audio library — no need to understand sound card hardware
A browser needs to display text	It calls the OS font rendering library — no need to draw every letter pixel by pixel

This saves enormous amounts of time for developers and ensures that common functions work consistently across all apps.

Device Drivers: The Hardware Translators

Every piece of hardware — your printer, your keyboard, your graphics card, your webcam — speaks a slightly different electrical language. A keyboard from one manufacturer works differently at the circuit level from a keyboard from another manufacturer.

A **device driver** is a special program that lets the OS communicate with a specific hardware device.

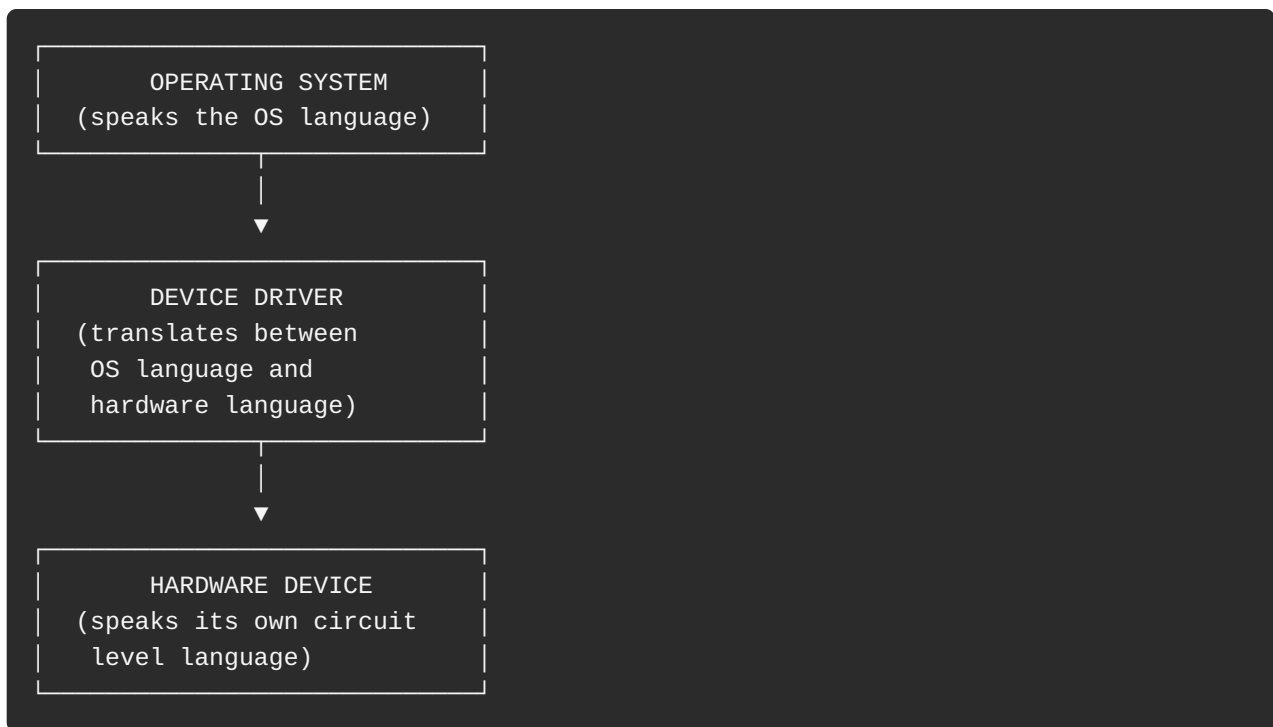


Figure 1.4: A device driver sits between the OS and the hardware, acting as a translator.

Examples of device drivers:

- **Printer driver** — Tells the OS how to send a document to a specific printer model
- **Graphics card driver** — Tells the OS how to render images and video using the GPU
- **Keyboard driver** — Interprets keystrokes from your specific keyboard model
- **Wi-Fi driver** — Allows the OS to send and receive data through your wireless chip

Have you ever seen this message? "Windows has found new hardware and is installing drivers." That is the OS automatically finding and installing the correct translator for a new device you just connected.

Practical Walkthrough: Viewing Device Drivers on Your Computer

On Windows:

1. Right-click the **Start** button.
2. Click **Device Manager**.
3. A list appears showing all the hardware categories in your computer.
4. Click the arrow next to **Display adapters** to expand it.
5. You will see your graphics card listed (e.g., "Intel HD Graphics" or "NVIDIA GeForce").
6. Right-click on it and select **Properties**.
7. Click the **Driver** tab.
8. You will see the driver version, date, and the manufacturer who wrote it.

What just happened? You looked at the translator program that allows Windows to communicate with your graphics chip. This driver was written by Intel, NVIDIA, or AMD — whoever made your GPU.

Pause & Think — 1.2.3

You buy a brand-new printer and connect it to your computer. The computer does not recognise it.

Why not? (Think about what is missing.)

What do you need to install?

Where do you usually find this software?

What happens if you install the wrong driver — for example, a driver written for a different printer model?

Write down your answers. Check with a classmate.

Quick Recap

System libraries provide ready-made code for common tasks so developers do not have to rebuild everything from scratch. **Device drivers** are translators that allow the OS to communicate with specific hardware. Both are essential connective tissue between software and the physical world.

1.3 Process Management in Operating System (OS)

1.3.1 Process Life Cycle

Story Mode: Margaret Hamilton and the Moon Landing

It is 20 July 1969. The Apollo 11 lunar module is 3,000 metres above the Moon's surface and descending. Suddenly, the onboard computer starts flashing an error code — 1202. An alarm.

The computer was overloaded. Too many tasks were running at the same time. Mission Control in Houston held its breath.

But nothing crashed.

The computer's operating system — designed by **Margaret Hamilton** and her team at MIT — had been built to handle exactly this situation. When overloaded, the OS automatically identified which tasks were *most critical* (landing the spacecraft) and which were *less critical* (navigation data collection), and it dropped the lower-priority tasks to protect the landing.

The Apollo 11 mission succeeded. Armstrong and Aldrin landed safely.

The lesson: an operating system that correctly manages processes under pressure can make the difference between success and catastrophe.

What is a Process?

When you open a program — say, a music player — the OS does not just "run it." It creates a **process**.

A **process** is a program that is currently loaded into memory and being executed by the CPU.

Every open app, every background task, every system service is a process. Right now, your computer or phone likely has dozens of processes running simultaneously.

The Process Life Cycle

Every process goes through a series of **states** during its life:

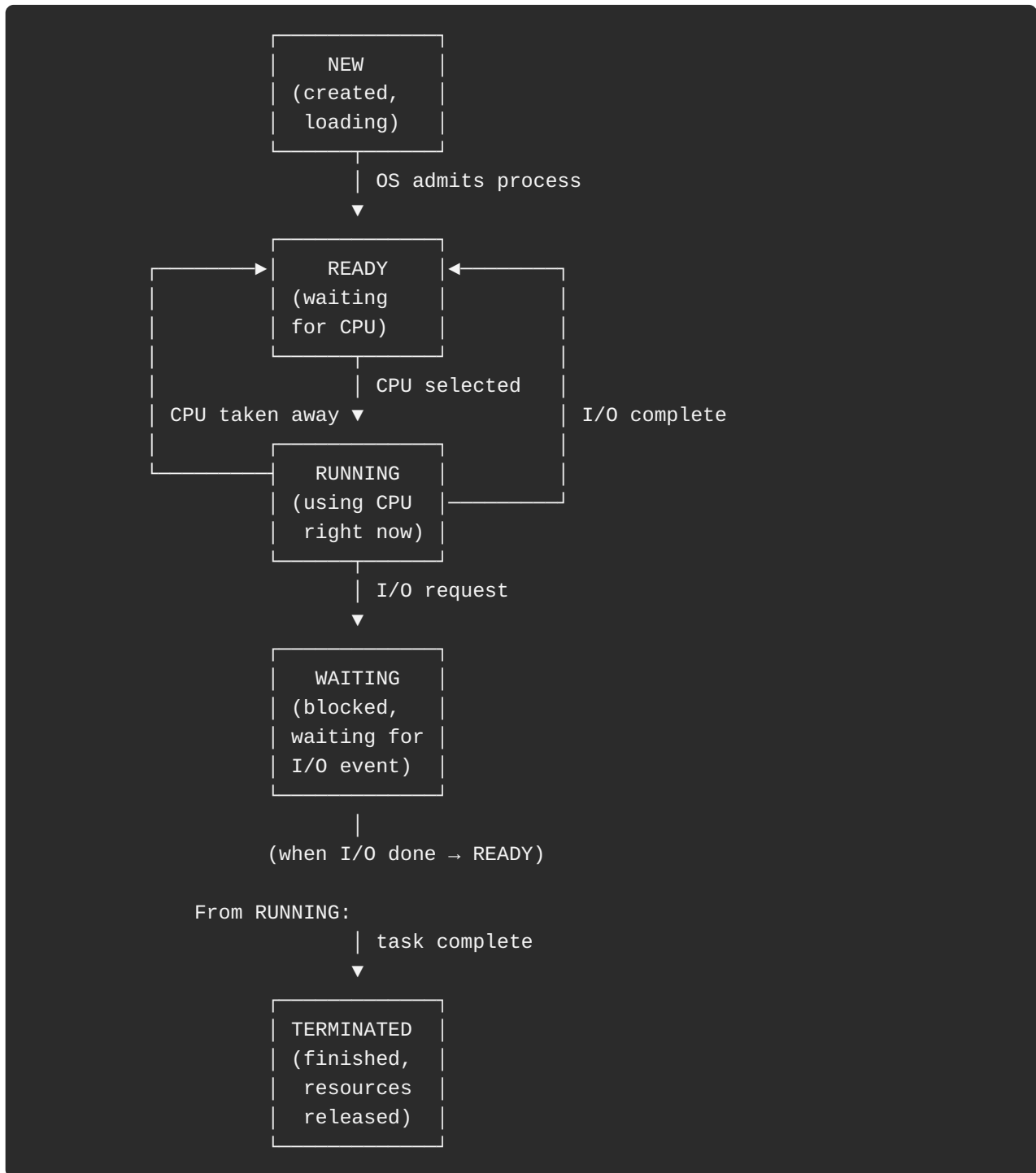


Figure 1.5: The Process State Diagram. A process moves through these states during its lifetime.

The Three Main Stages: Creation, Execution, Termination

1. Creation

When you double-click an application icon:

- The OS reads the program's file from the storage drive
- It loads the program's instructions into RAM
- It creates a **process entry** — a record in the OS's table that tracks this process
- The process enters the **NEW** state and then moves to **READY**

2. Execution

- The process moves to the **RUNNING** state when the CPU begins working on it
- While running, the process might request data from a file or wait for network information — this moves it to the **WAITING** state
- Once the data arrives, it returns to **READY**, then gets CPU time again

3. Termination

- When the process finishes its work (or you close the app), it enters the **TERMINATED** state
- The OS releases all the memory and resources the process was using
- Those resources become available for other processes

Real Example: Opening Google Chrome

Phase	What Happens
Creation	You double-click the Chrome icon. OS loads Chrome's code from the hard drive into RAM. A new process (e.g., Process ID #1234) is created.
Execution	Chrome runs — it loads your home page, renders text, plays animations.
Waiting	Chrome waits for a webpage to respond from the internet. During this wait, the CPU works on other processes.
Ready → Running	The webpage responds. Chrome's process is ready again and gets CPU time to display the page.
Termination	You click the X to close Chrome. The OS closes the process, frees the RAM it was using, and the memory becomes available for other programs.

Pause & Think — 1.3.1

You are listening to a music app. While the music plays, a phone call comes in. You answer the call.

Trace through the process states for each app:

- What happens to the **music app** process when the call comes in?

- What state is the **music app** in while you are on the call?
- What happens to the **music app** when you end the call and return to it?
- What state is the **phone call app** in during the call?

Draw the state transitions for both processes side by side.

Quick Recap

A **process** is a running program managed by the OS. Every process moves through states: New → Ready → Running → Waiting → Terminated. The OS tracks every process and manages what resources each one gets. Understanding this lifecycle explains why apps "load," "freeze," "resume," and "close."

1.3.2 Multitasking and Concurrency

Story Mode: The Master Chef

A master chef is preparing five dishes for a dinner party. The chef cannot cook five things at the same time on one stove burner. But to the guests sitting at the table, it *appears* that everything is being prepared simultaneously. How?

The chef works on the soup for two minutes, moves to the salad for one minute, stirs the pasta for thirty seconds, checks the bread, returns to the soup, and so on — cycling through tasks so rapidly that all five dishes are ready at roughly the same time.

Your CPU does exactly this — switching between processes so fast that you never notice.

Multitasking

Multitasking means the OS allows more than one program to be open and usable at the same time.

From your perspective, music plays *while* you type *while* messages arrive *while* a file downloads.

In reality, on a single CPU core, only **one process runs at any given instant**. But the OS switches between processes so rapidly — thousands of times per second — that everything *appears* to run simultaneously.

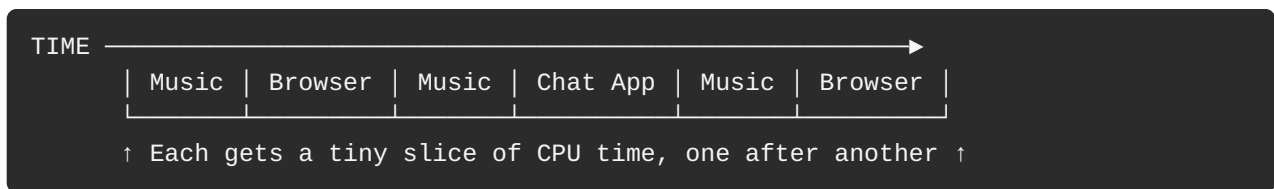


Figure 1.6: Time-slicing. The CPU rapidly switches between processes, giving each a small "slice" of time.

Concurrency

Concurrency means multiple processes are *active* (started and not yet finished) at the same time — even if only one is actually using the CPU at any instant.

Think of it this way:

- Multiple pots are on the stove — all *in progress* (concurrent)
- But only one burner is actively heating at any moment (CPU)

Concurrency is what allows your computer to have 30 things open without completing them all before starting something new.

Multitasking vs. Concurrency — A Simple Comparison

Concept	Meaning	Analogy
Multitasking	OS lets multiple programs be active and usable at once	Many apps open in your taskbar
Concurrency	Multiple processes are in-progress simultaneously, even if CPU serves only one at a time	Chef cooking five dishes, working on each in rotation

Pause & Think — 1.3.2

Your computer has one CPU core. You have these four things running:

- A music player
- A word processor (you are actively typing)
- A file download (downloading in the background)
- An antivirus scan (running quietly)

Which process do you think deserves the most CPU time?

Which process probably needs only occasional CPU attention?

In what order should the OS prioritise these tasks to give you the best experience?

Explain your reasoning. Then compare with a classmate.

Quick Recap

Multitasking lets you have multiple apps open and usable at once. **Concurrency** means many processes are in-progress at the same time, even though the CPU can only work on one at a time. The OS switches between them so rapidly that everything feels simultaneous.

1.3.3 Process Scheduling Concepts

The Problem: One CPU, Many Waiting Processes

Imagine you run a single help desk at school. Ten students are waiting in line, each with a different question. Some questions take 30 seconds. Some take 10 minutes. You can only help one student at a time.

How do you decide who goes first?

This is exactly the challenge the OS faces. This decision-making process is called **scheduling**.

Scheduling is the method the OS uses to decide:

1. Which process gets CPU time next
 2. How long each process runs before being switched out
-

First-Come, First-Served (FCFS) Scheduling

The simplest scheduling method is **First-Come, First-Served (FCFS)**.

Rule: The process that *arrived first* gets the CPU first. Each process runs to completion before the next one starts. No jumping the queue.

Analogy: A shop with a single checkout counter. Customers are served in the exact order they joined the queue.

Numerical Example: FCFS in Action

Let us say the CPU has three processes waiting:

Process ID	Arrival Time	Time Needed (Burst Time)
P1	0 seconds	5 seconds
P2	1 second	3 seconds
P3	2 seconds	2 seconds

Step-by-step walkthrough:

At 0 seconds:

- P1 arrives. No one else is waiting.
- P1 starts running immediately.
- P1 needs 5 seconds → will finish at second 5.

At 1 second:

- P2 arrives. P1 is still running. P2 joins the queue and waits.

At 2 seconds:

- P3 arrives. P1 is still running. P2 is still waiting. P3 goes behind P2 in the queue.

At 5 seconds:

- P1 finishes. CPU is free.
- P2 is next in the queue. P2 starts running.
- P2 needs 3 seconds → will finish at second 8.

At 8 seconds:

- P2 finishes.
- P3 is next. P3 starts running.
- P3 needs 2 seconds → will finish at second 10.

CPU Timeline Diagram

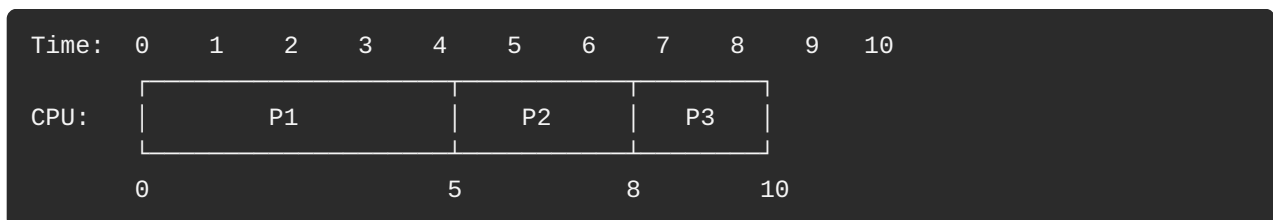


Figure 1.7: FCFS execution timeline. P1 runs first (0–5s), then P2 (5–8s), then P3 (8–10s).

Waiting Time Calculation

Waiting time = Time a process spent waiting before the CPU started working on it.

Process	Arrival Time	Start Time	Waiting Time
P1	0s	0s	0s (started immediately)
P2	1s	5s	$5s - 1s = 4 \text{ seconds}$
P3	2s	8s	$8s - 2s = 6 \text{ seconds}$

Notice: **P3 arrived at 2 seconds but waited 6 full seconds** before the CPU even started working on it — even though P3 was the *shortest* task (only 2 seconds of work needed).

This is the main weakness of FCFS: short tasks can be stuck behind long ones.

Advantages and Disadvantages of FCFS

Advantages	Disadvantages
Very simple — easy to understand and code	Short tasks can wait a very long time behind long tasks
Fair — everyone waits their turn	Poor performance when task lengths vary greatly
No task is ever completely ignored	Not suitable for real-time systems where speed matters
Predictable — you know exactly when you will be served	Can cause the "convoy effect" — short tasks stuck behind one long task

The Convoy Effect: Imagine a slow lorry on a single-lane road. All the fast cars behind it are stuck. In FCFS, a long process is like that slow lorry — all shorter processes stack up behind it.

Practical Walkthrough: Watch Real Processes on Your Computer

On Windows:

1. Press **Ctrl + Shift + Esc** to open **Task Manager**.
2. Click the **Processes** tab.
3. You can see all currently running processes, their CPU usage (%), and memory usage.
4. Click the **CPU** column header to sort by CPU usage. The process using the most CPU rises to the top.
5. Watch the numbers change in real time — the OS is scheduling these processes right now, giving each CPU time.

On macOS:

1. Open **Finder** → **Applications** → **Utilities** → **Activity Monitor**.
 2. Click the **CPU** tab to see all running processes and their CPU usage.
-

Pause & Think — 1.3.3

Using the FCFS method, solve this scheduling problem:

Process	Arrival Time	Time Needed
P1	0s	3s
P2	1s	5s
P3	2s	3s

1. Draw the CPU timeline (like the diagram above).
 2. Calculate the waiting time for each process.
 3. Which process waited the longest?
 4. Is it fair that P3 — which only needs 3 seconds — has to wait so long?
 5. Can you think of a better rule that might help short processes get served faster?
-

Quick Recap

Scheduling is the OS deciding which process gets CPU time next and for how long. **FCFS (First-Come, First-Served)** is the simplest method: first to arrive, first to be served. It is fair and easy, but short tasks can get stuck behind long ones — called the convoy effect.

1.4 Memory

1.4.1 Primary Memory (RAM)

Story Mode: The Desk and the Filing Cabinet

Picture yourself studying for an important exam. Your bedroom has a large filing cabinet full of textbooks, notes, and past papers. But you can only work on what is on your desk right now.

When you need the history textbook, you take it from the filing cabinet and put it on your desk. When you need a calculator, you put it on the desk too. The desk is your *active workspace*. The filing cabinet is your *long-term storage*.

But your desk has a limited size. If you put too many things on it, everything gets cramped and you cannot work efficiently.

In a computer:

- The **desk** is **RAM (Random Access Memory)** — fast, temporary, currently-in-use storage
 - The **filing cabinet** is your **hard drive or SSD** — slow, permanent, long-term storage
-

What is RAM?

RAM stands for **Random Access Memory**. It is the computer's primary (main) working memory.

Key characteristics of RAM:

Property	Description
Speed	Extremely fast — the CPU can read from RAM billions of times per second
Temporary	Data is erased when the computer is turned off (it is "volatile")
Limited size	Typically 4 GB, 8 GB, or 16 GB on modern computers
Purpose	Holds the programs and data that are <i>currently active</i>

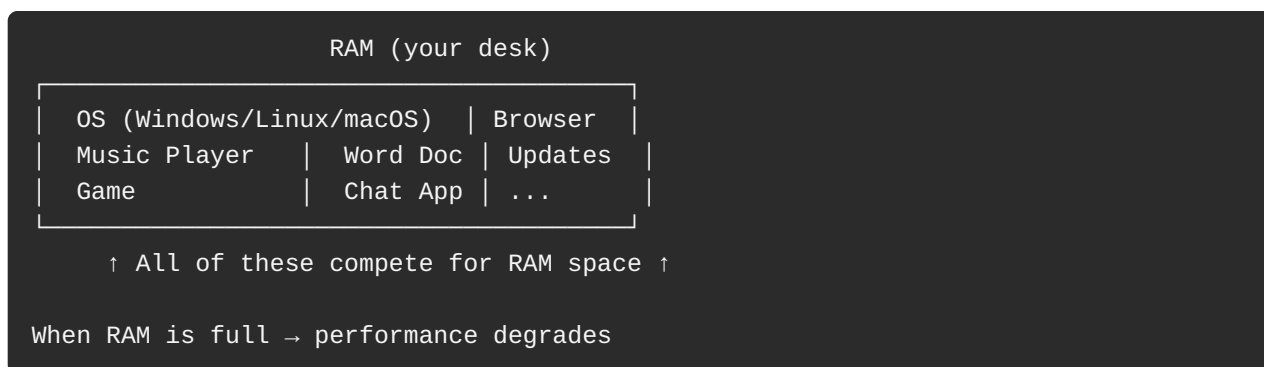
What does RAM hold right now on your computer?

- The operating system itself
 - Every open application (browser, music player, document editor)
 - The document you are typing
 - The webpage you are viewing
 - Variables and data that running programs are currently using
-

Why RAM Matters for Performance

More RAM → more programs can be open at once → smoother performance.

Less RAM → the OS struggles to keep everything active → programs freeze or slow down.



Fascinating Fact: The IBM 5150, the first popular personal computer (1981), came with only **16 kilobytes** of RAM. A single photo on your smartphone today is larger than the total RAM of that machine. Modern computers can have DDR5 RAM capable of transferring over **50 gigabytes per second**.

Practical Walkthrough: Check Your Computer's RAM

On Windows:

1. Press **Windows key + Pause/Break**, or right-click **This PC** and select **Properties**.
2. Under **System**, look for **Installed RAM**.
3. You will see something like: "8.00 GB"

Or, to see RAM *in use* right now:

1. Press **Ctrl + Shift + Esc** to open Task Manager.
2. Click the **Performance** tab.
3. Click **Memory**.
4. You will see a real-time graph of how much RAM is currently in use.

What just happened? You are looking at your computer's "desk size" and seeing how full it currently is.

Pause & Think — 1.4.1

Your computer has 4 GB of RAM. You have these programs open:

- Web browser with 15 tabs: uses approximately 2 GB
- Music player: uses approximately 200 MB
- Word processor: uses approximately 300 MB
- Game: requires 3 GB to run

Can you open the game without closing anything?

Which program(s) would you close to free up enough space?

What would happen if you tried to open the game anyway?

Calculate the total RAM needed and determine what is possible.

Quick Recap

RAM is the computer's fast, temporary working memory. Everything currently running — OS, apps, files you are editing — lives in RAM. More RAM means more can run at once. RAM is erased when the computer powers off; it is a workspace, not a permanent storage place.

1.4.2 Virtual Memory

Story Mode: The Manchester Atlas and the Magic Trick

It is 1962 at the University of Manchester, England. Researchers are building the **Atlas computer** — one of the most advanced machines of its era.

They face a problem: programs are getting too large to fit in the computer's physical memory. Programs would crash the moment they exceeded the memory limit.

A team led by Tom Kilburn and his colleagues invented a clever solution: they would create the *illusion* of more memory than physically existed.

When a program needed more space than RAM could hold, the OS would silently swap parts of the program to a section of the hard drive — and bring them back when needed. To the program, it appeared as if it had unlimited memory. In reality, the OS was constantly juggling data between fast RAM and slower disk storage.

They called this **virtual memory** — memory that seems to exist beyond what is physically there. It is still the technique used in every computer and smartphone today.

What is Virtual Memory?

Virtual memory is a technique where the OS uses a portion of the **storage drive** (hard drive or SSD) as extra RAM when physical RAM is full.

The analogy revisited:

Imagine you run out of desk space. You cannot fit any more books. So you put some books in a backpack on the floor next to your desk. When you need a book from the backpack, you pick it up and put it on the desk (moving something else to the backpack to make room).



Figure 1.8: Virtual memory acts as an extension of RAM. When RAM is full, the OS moves less-used data to the storage drive.

How Virtual Memory Works

1. You have 8 GB of RAM, and you open so many programs that 8 GB fills up.
2. The OS looks at which programs you have not used recently.
3. It takes those programs' data from RAM and copies it to a special reserved area on the hard drive called the **page file** (Windows) or **swap space** (Linux/macOS).
4. This frees up RAM for the programs you *are* actively using.
5. When you switch back to the program that was moved out, the OS copies its data back from the drive into RAM (possibly moving something else out to make room).

This is called paging or swapping.

The Trade-Off: Virtual Memory is Slower

Hard drives and SSDs are **much slower** than RAM.

Storage Type	Approximate Speed
RAM (DDR5)	~50,000 MB per second
NVMe SSD	~7,000 MB per second
Regular SSD	~500 MB per second
Hard Drive	~100–150 MB per second

When your OS starts heavily using virtual memory (called **thrashing**), you will notice:

- Programs take much longer to respond
- The hard drive makes constant clicking or grinding sounds (on older machines)
- Everything feels sluggish

This is why adding more physical RAM always improves performance more than relying on virtual memory.

Practical Walkthrough: Check Virtual Memory Settings on Windows

1. Press **Windows key**, type **System**, and open **System** settings.
2. Click **Advanced system settings** (left panel or link).
3. Under **Performance**, click **Settings**.
4. Click the **Advanced** tab.
5. Under **Virtual memory**, click **Change**.
6. You will see the current paging file (virtual memory) size.

Note the size listed. This is the amount of hard drive space Windows has reserved to use as extra RAM when needed.

Pause & Think — 1.4.2

Imagine your computer has 4 GB of RAM. You are running a video editing project that needs 6 GB.

Without virtual memory: What would happen when the program tries to use more than 4 GB?

With virtual memory: How does the OS allow the program to continue running?

Why is the video editing still slower than on a computer with 8 GB of physical RAM, even with virtual memory enabled?

Write a short paragraph explaining the trade-off.

Quick Recap

Virtual memory uses part of the storage drive as extra RAM when physical RAM is full. It allows more programs to run simultaneously than the RAM alone could support. However, because hard drives are much slower than RAM, using virtual memory comes at a performance cost — the system slows down.

1.5 Processes vs. Threads

Story Mode: The Restaurant Kitchen

A restaurant has one large kitchen (a process). Inside, multiple cooks work simultaneously (threads):

- Cook 1 is chopping vegetables
- Cook 2 is grilling meat
- Cook 3 is preparing sauces
- Cook 4 is plating finished dishes

All cooks share the same kitchen, the same tools, and the same refrigerator. They are all working on the same meal order. They are all part of the same "process" — preparing your dinner — but each is doing a different sub-task independently.

If the whole kitchen closes (process terminates), all cooks stop. If one cook takes a break (thread pauses), the others continue.

Process vs. Thread — The Core Difference

	Process	Thread
Definition	An independent running program	The smallest unit of execution inside a process
Memory	Has its own private memory space	Shares memory with other threads in the same process
Independence	Isolated — one process crashing does not affect others	If one thread crashes badly, it can affect the whole process
Resource cost	Expensive to create — requires its own memory allocation	Cheap to create — shares the parent process's resources
Communication	Communicating between processes is complex	Threads in the same process share data easily
Example	Two separate apps: Chrome and Word	Two tabs inside the same Chrome window

The Web Browser Example

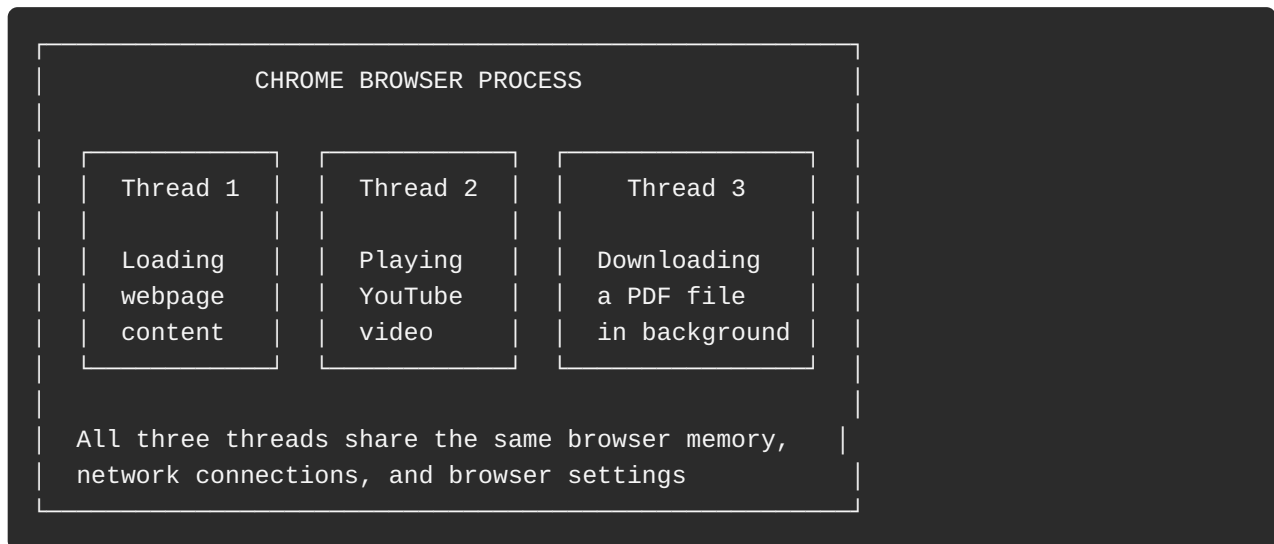


Figure 1.9: One browser process containing multiple threads, each handling a different task simultaneously.

While Thread 2 plays the video, Thread 1 continues loading the page, and Thread 3 keeps downloading the file. You do not have to wait for the video to finish before the download completes.

Pause & Think — 1.5

Compare these two scenarios:

Scenario A: You open 10 browser tabs inside one Chrome window.

Scenario B: You open 10 completely separate Chrome windows.

In Scenario A: All tabs are threads inside one process. They share Chrome's memory.

In Scenario B: Each window might be a separate process. Each one has its own memory allocation.

Questions:

- Which scenario uses more RAM? Why?
 - If one tab crashes in Scenario A, what might happen to the other tabs?
 - If one window crashes in Scenario B, what happens to the other windows?
 - Which approach is more resource-efficient?
-

Quick Recap

A **process** is a complete, independent running program with its own memory. A **thread** is a smaller unit of execution inside a process, sharing the process's memory. Multiple threads can work simultaneously within one process, making programs faster and more responsive.

1.5.1 Multithreading

What is Multithreading?

Multithreading is the ability of a single process to run multiple threads at the same time.

Instead of doing tasks one by one (finish spelling check, *then* let you type), a multithreaded program does many things concurrently (spell check runs on one thread *while* you type on another thread, simultaneously).

WITHOUT MULTITHREADING:

```
|— Type text —|— Check spelling —|— Save file —|  
Tasks done one at a time (sequential)
```

WITH MULTITHREADING:

```
|— Type text —————|  
|— Check spelling (background) —————|  
|— Auto-save (background) —————|  
All three happen at the same time (parallel / concurrent)
```

Figure 1.10: Multithreading allows a program to do multiple tasks simultaneously instead of waiting for each to finish.

1.5.2 Benefits of Multithreading

1. Enhanced Performance

Complex tasks can be divided into sub-tasks and run in parallel.

Example: A video game uses threads for:

- Graphics rendering (drawing the game world)
- Physics simulation (calculating object movements)
- Sound playback (music and sound effects)
- AI logic (enemy decision-making)
- Network synchronisation (multiplayer data)

Without multithreading, the game would have to finish drawing every frame *before* calculating physics, *before* playing sound — making it painfully slow.

2. Improved Responsiveness

An application stays interactive (responsive to your clicks and typing) even while doing heavy background work.

Example: Microsoft Word

- Thread 1: Accepts your keyboard input (so you can keep typing)
- Thread 2: Runs spell check and grammar check in the background
- Thread 3: Periodically auto-saves the document

You never feel the spell check "freeze" your typing because it runs on a separate thread.

3. Support for Concurrent Operations

Multiple long-running operations can overlap.

Example: A video editing application

- Thread 1: Plays back the preview of your video edit
 - Thread 2: Exports/renders a finished video in the background
 - Thread 3: Loads the next section of footage from the disk
-

4. Efficient Use of Resources

Threads are lighter than processes. Creating a new thread inside a process is much faster and uses much less memory than creating an entirely new process.

Summary Table: Benefits of Multithreading

Benefit	What It Means	Real Example
Performance	Parallel execution speeds up complex tasks	Game rendering + physics + sound simultaneously
Responsiveness	UI stays interactive during heavy work	Typing while spell check runs in background
Concurrency	Multiple operations overlap	Downloading + displaying + saving at the same time
Efficiency	Threads share memory, so less overhead	Browser tabs use less RAM than separate browser windows

Grab a Partner — 1.5.2

Think of a photo editing application (like a mobile camera filter app).

Partner A: List 3 things the app might be doing at the same time when you apply a filter to a photo.

Partner B: For each of those 3 things, decide: *Would you make it a thread? Or a separate process? Why?*

Discuss whether these tasks should share memory (suggest threads) or be isolated (suggest processes).

Quick Recap

Multithreading allows a single program to do multiple things at once by dividing its work into threads. Benefits include faster performance, better responsiveness, support for concurrent operations, and efficient use of memory. Almost every modern application — browsers, word processors, games — uses multithreading.

1.6 System Calls

Story Mode: The Catastrophic Early Days of Personal Computing

In the early 1980s, personal computers ran an operating system called **MS-DOS**. It had almost no security layer between programs and hardware. Any application could directly access memory, write to the hard drive, or send instructions to the CPU.

This caused disasters. A badly written game could accidentally overwrite operating system data, crashing the entire machine. A malicious program could read data it was never supposed to touch. Some viruses of that era simply overwrote the hard drive's boot sector — the critical area that tells the computer how to start — making the computer completely unbootable.

The solution, gradually implemented in modern operating systems, was a **controlled gateway**: no program could touch the hardware directly. Instead, every program had to make a formal request to the OS and ask it to perform hardware operations on its behalf.

This formal request is called a **system call**.

What is a System Call?

A **system call** is a formal request that a program makes to the operating system, asking it to perform a hardware-level task that the program is not allowed to do directly.

System calls are the **only legal gateway** between user programs and the OS kernel.

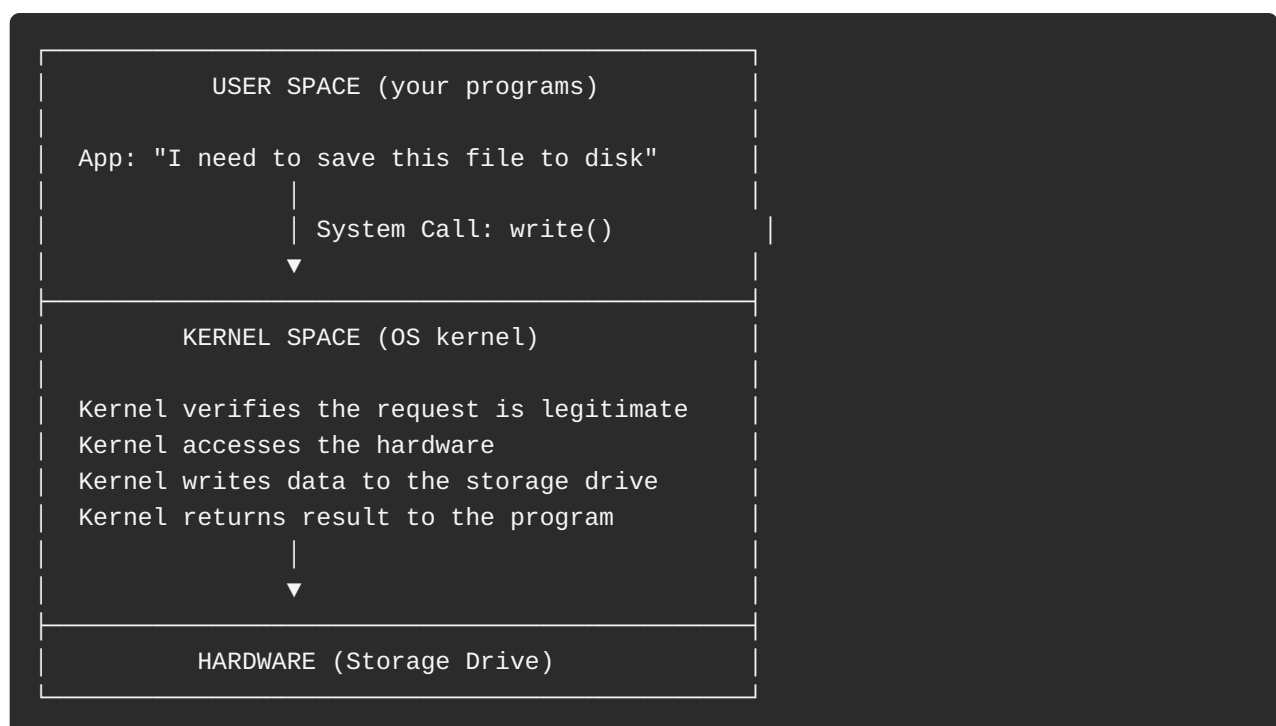


Figure 1.11: A system call crosses the boundary from user space into kernel space, where the OS performs the hardware action safely.

Why are system calls necessary?

1. **Security:** Programs cannot directly access or corrupt hardware, memory, or other programs' data
 2. **Stability:** The OS can validate requests before acting — if a program asks to write to an invalid memory location, the OS rejects it
 3. **Fairness:** The OS mediates hardware access so multiple programs share devices fairly
-

1.6.1 Types of System Calls

There are four primary types of system calls you need to know:

1. `open()`

Purpose: Opens a file for reading or writing.

What it does: Tells the OS: "I want to access this file. Please find it on the storage drive and give me a connection to it."

Example: When you open a music file in your music player, the application calls `open("song.mp3")`. The OS locates the file, checks permissions (are you allowed to access this file?), and returns a file handle — a reference the app can use to read data.

2. `read()`

Purpose: Reads data from a file or input device.

What it does: Once a file is open, `read()` retrieves its contents and passes them to the application.

Example: After `open("document.txt")`, the word processor calls `read()` to load the text content from disk into RAM so it can be displayed on your screen.

3. `write()`

Purpose: Sends data to a file or output device.

What it does: Saves data from a program's memory to a file on disk, or sends data to an output device (like a speaker or printer).

Example: When you press Ctrl+S to save your document, the word processor calls `write()` with the document data. The OS handles the actual physical writing to the hard drive.

4. `fork()`

Purpose: Creates a new process by duplicating an existing one.

What it does: When a program needs to create a child process — an independent sub-process to handle a separate task — it uses `fork()`. The OS creates an exact copy of the calling process.

Example: When you open a new tab in Chrome, the OS may use `fork()` to create a new process (or thread) to handle that tab independently.

System Call Summary Table

System Call	Action	Everyday Example
<code>open()</code>	Opens a file	Opening a music file to play
<code>read()</code>	Reads data from file or device	Loading a document's text
<code>write()</code>	Writes data to file or device	Saving a photo to your computer
<code>fork()</code>	Creates a new process	Opening a new browser tab

Did You Know? Linux and macOS have a few hundred system calls each. Windows uses nearly **2,000** different system calls. These calls handle everything from opening files and drawing graphics to managing network connections and running security checks — all happening silently as you use your computer.

Pause & Think — 1.6.1

Think about this sequence of events when you take a photo on your phone and send it to a friend:

1. Camera app captures the image
2. Image is saved to your photo library
3. You open the messaging app
4. You attach and send the photo

For each step, identify which system call(s) might be involved:

- Capturing and writing the image to storage → ??
- Opening the messaging app → ??
- Reading the photo file to attach it → ??
- Writing the photo to the outgoing message → ??

Match each step to one or more of: `open()`, `read()`, `write()`, `fork()`

Quick Recap

A **system call** is a formal request a program makes to the OS to perform a hardware-level action. System calls are the only safe way for programs to access hardware — they go through the OS kernel, which validates and controls every request. The main types are `open`, `read`, `write`, and `fork`.

1.7 File System Structure and Management

Story Mode: The Library Before Card Catalogues

Imagine a library with 100,000 books — but no shelves, no labels, no ordering system. Books are piled randomly on the floor. When you want *Chemistry for Class 10*, you have to physically pick up and check every single book.

Now imagine the library gets a brilliant librarian. She builds labelled shelves (folders), organises books by subject and class (hierarchy), creates a card catalogue that lists every book, its location, its author, its date of acquisition, and its size (metadata). Now finding any book takes seconds.

A computer's file system is that brilliant librarian — organising millions of files so that you, and the OS, can find anything in an instant.

1.7.1 Files

A **file** is a collection of related data stored on a computer.

A file can contain:

- **Text** (a story, an essay, code)
- **Images** (photos, icons, diagrams)

- **Audio** (music, voice recordings)
- **Video** (movies, tutorials)
- **Program instructions** (executable files like .exe)

Every file has:

- A **name** (e.g., `homework.docx`)
- An **extension** (`.docx` tells the OS this is a Word document)
- A **location** (which folder it lives in)
- **Content** (the actual data stored inside)

1.7.2 Folders

A **folder** (also called a **directory**) is a container that holds files and other folders.

Folders create a **hierarchical** (tree-like) structure for organising files:

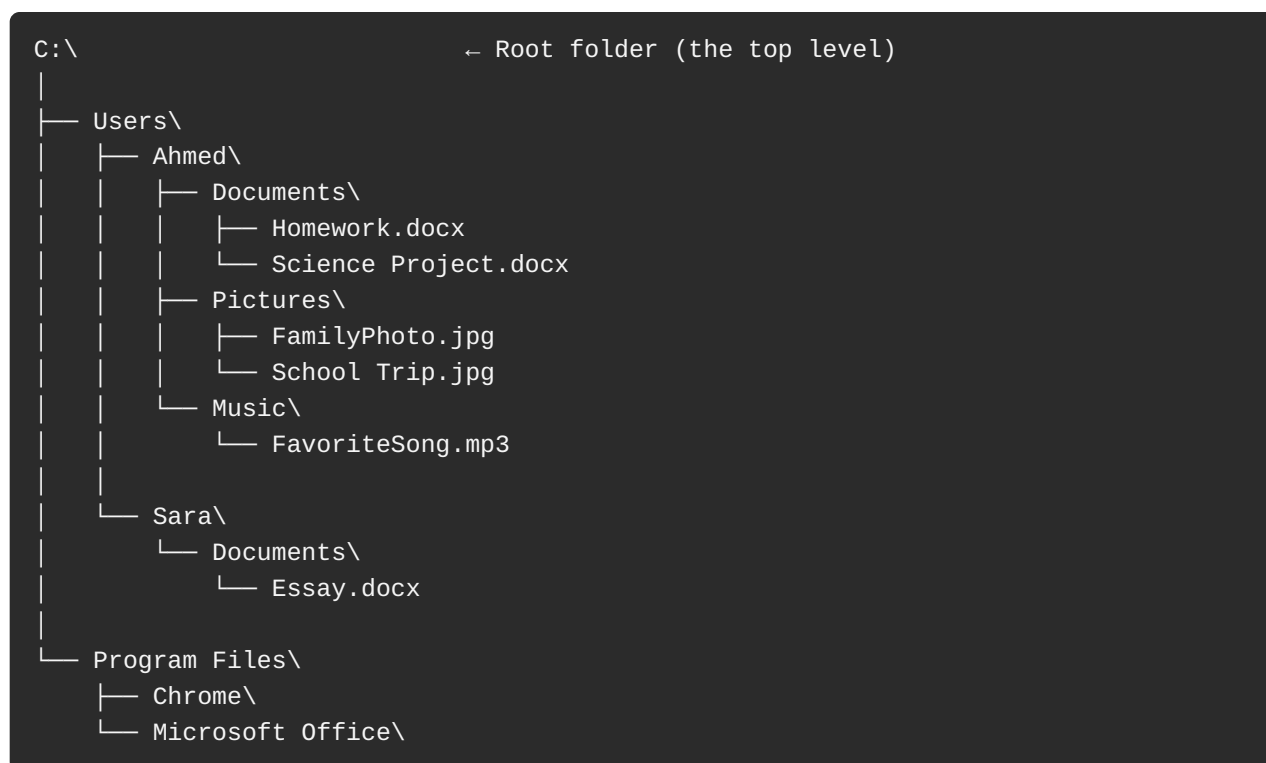


Figure 1.12: A typical file system hierarchy on a Windows computer. Folders contain files and other folders in a tree structure.

This tree structure means:

- Every file has a unique **path** — the address that leads to it
- Example: `C:\Users\Ahmed\Documents\Homework.docx`
- Folders within folders let you organise millions of files logically

1.7.3 Metadata

Metadata is *data about data* — information that describes a file without being the file's actual content.

Think of a physical book. The **content** is the text inside. The **metadata** is everything on the cover and spine: title, author, publisher, publication year, page count, subject category.

Examples of file metadata:

Metadata Field	Example
File name	holiday_photo.jpg
File type	JPEG Image
File size	3.2 MB
Date created	14 August 2025
Date last modified	15 August 2025
Dimensions (for images)	4032 × 3024 pixels
Location taken (for photos)	GPS coordinates
Author (for documents)	Ahmed Ali
Read/Write permissions	Owner: Read+Write; Others: Read only

Why does metadata matter?

- The OS uses metadata to quickly **find and sort files** without opening them
- File managers show metadata so you can identify files at a glance
- Security systems use **permissions** metadata to control who can access what
- Photo apps use GPS metadata to show where each photo was taken on a map

Practical Walkthrough: Viewing File Metadata

On Windows:

1. Find any file on your Desktop or in Documents.
2. **Right-click** on it.
3. Click **Properties**.
4. The Properties window shows: file size, dates created and modified, file type, and location.
5. Click the **Details** tab for even more metadata (especially rich for photos — you may see camera model, exposure settings, GPS location).

On macOS:

1. Right-click on any file.
 2. Click **Get Info**.
 3. A panel opens showing all available metadata.
-

Pause & Think — 1.7.3

Take out your smartphone. Go to your Photos app and open any photo.

Find the photo's details or information (usually by tapping the  icon or swiping up).

List **5 pieces of metadata** visible for that photo that are *not* visible in the image itself.

Consider: size, date, time, location, camera settings, file format, dimensions.

Why might it be important — or potentially dangerous — that a photo contains GPS location data?

1.7.4 File Systems

A **file system** is the method the OS uses to organise, store, and retrieve files on a storage device.

It is the invisible system that:

- Decides *where* on the physical disk each file goes
- Keeps track of which disk sectors belong to which file
- Manages the folder hierarchy
- Handles file permissions and access control

Why do different file systems exist?

Different devices have different needs. A USB drive needs a simple, widely compatible system. A professional server needs robust security and large file support. A smartphone needs efficiency and crash recovery.

File System	Used By	Key Features
FAT32	USB flash drives, older devices	Very compatible — works on almost any device; limited to 4 GB per file
NTFS	Windows computers	Large file support, detailed permissions, journaling (crash recovery)
APFS / HFS+	macOS (Apple computers)	Optimised for SSDs, strong encryption, fast metadata operations
EXT4	Linux computers	Widely used on Linux, excellent performance, journaling, large file support
exFAT	Modern USB drives, SD cards	Compatible with both Windows and Mac; no 4 GB file size limit

Analogy: Different countries use different road systems — some drive on the left, some on the right; some roads are dirt tracks, some are motorways. They all get you from A to B, but they have different rules and are designed for different needs. File systems are the road systems of digital storage.

Pause & Think — 1.7.4

You format a USB drive with FAT32 to use it on your home computer. Later, you try to copy a 5 GB video file onto it.

What happens? Why?

Which file system would you use instead?

What would you need to do to the USB drive before it could accept the 5 GB file?

Quick Recap

The **file system** organises how files are stored on a disk. **Files** contain data. **Folders** organise files hierarchically. **Metadata** describes files without being their content. Different file systems (FAT32, NTFS, EXT4, APFS) serve different needs. Together, these structures allow you to store and retrieve millions of files reliably and quickly.

1.8 Types of Operating Systems

Overview

Not every device needs the same kind of operating system. A space satellite has radically different needs from a smartwatch, which has different needs from a school server, which has different needs from your smartphone.

Operating systems are therefore designed for specific environments. Let us explore four major types.

1.8.1 Real-Time Operating System (RTOS)

Story Mode: The Pacemaker and the Deadline

A **pacemaker** is a small device implanted in a person's chest. Its job is to monitor the heart and deliver a tiny electrical pulse when the heart misses a beat — restoring the correct rhythm.

This device runs an operating system. But unlike your phone's OS, which can afford to pause for a few milliseconds while it checks for app updates, the pacemaker's OS has a non-negotiable rule: **the pulse must be delivered within an exact time window**. A delay of even 50 milliseconds could mean a skipped heartbeat. A delay of a second could be fatal.

This is a **real-time** requirement: the response must happen within a guaranteed time limit, no exceptions.

What is an RTOS?

A **Real-Time Operating System (RTOS)** is an OS designed to process tasks and deliver results **within strict, guaranteed time limits** called **deadlines**.

Key characteristic: In an RTOS, *when* a task is completed is as important as *whether* it is completed correctly. A correct answer delivered too late is a failure.

Differences from General-Purpose OS:

General-Purpose OS (Windows, macOS)	Real-Time OS
Response time may vary — sometimes slow	Response time is guaranteed and predictable
Optimises for overall system throughput	Optimises for meeting every deadline
Acceptable to miss a deadline occasionally	Missing a deadline is a critical failure
Used on PCs, phones, tablets	Used in medical, industrial, aerospace systems

Where RTOS is used:

Application	Why Real-Time Matters
Air traffic control systems	Radar must update within milliseconds to track fast-moving aircraft
Heart monitors and pacemakers	Heartbeat response must occur within exact time windows
Industrial robot arms	Motor commands must be timed precisely or the arm moves incorrectly
Anti-lock braking systems (ABS) in cars	Brakes must respond within milliseconds during a skid
Missile guidance systems	Trajectory corrections must be computed and applied in real time

Pause & Think — 1.8.1

A self-driving car uses an RTOS to control braking. A pedestrian steps in front of the car. The car's sensor detects the obstacle.

If the RTOS cannot guarantee a response within 50 milliseconds, what could happen?

Why would a general-purpose OS like Windows be dangerous in this application, even though Windows is more "powerful" in terms of features?

Quick Recap

An **RTOS** processes tasks within guaranteed time limits (deadlines). Used in life-critical systems — medical devices, aircraft, industrial robots — where a late response is as dangerous as a wrong response. Speed and predictability take priority over features and flexibility.

1.8.2 Embedded Operating System (EOS)

Story Mode: The Computer Inside Your Microwave

Pick up your household microwave oven. It has a small keypad, a digital display, and it heats food by generating precisely controlled microwave radiation.

Inside that microwave is a tiny computer running a tiny operating system. It does not run Chrome. It cannot play music. It has no desktop wallpaper. It does exactly **one job**: controlling the microwave's heating cycles, managing the timer, and responding to the keypad.

That tiny, purpose-built OS is an embedded operating system.

What is an EOS?

An **Embedded Operating System** is a small, highly efficient OS built permanently into a specific device to perform a fixed, limited set of tasks.

Key characteristics:

- Very small — takes up minimal memory (sometimes as little as a few kilobytes)
- Very efficient — uses very little power
- Very focused — does only what the device needs, nothing else
- Often stored in **ROM (Read-Only Memory)** — cannot be easily changed

Comparison:

General-Purpose OS	Embedded OS
Runs thousands of different applications	Runs one specific application forever
Large — several gigabytes of storage needed	Tiny — may need only kilobytes
Updates are common and expected	Rarely (if ever) updated
High power usage	Designed for minimal power usage
Used on phones, computers, tablets	Used in appliances, machines, small devices

Where EOS is used:

Device	What the Embedded OS Does
Microwave oven	Manages timer, temperature control, keypad input
Washing machine	Controls wash cycles, water levels, spin speeds
ATM machine	Manages cash dispensing, receipt printing, card reading
Smart TV	Controls channel management, remote input, display
Printer	Manages print jobs, paper feeding, ink levels
Digital camera	Controls image capture, storage, display

Pause & Think — 1.8.2

You are designing an embedded OS for a digital alarm clock. The clock needs to:

- Display the current time
- Sound an alarm at a set time
- Allow the user to set and cancel alarms

What features of a general-purpose OS (like Windows) would you NOT need?

What is the most important thing your embedded OS must always do reliably?

What would happen if your alarm clock's OS crashed?

Quick Recap

An **Embedded OS** is a tiny, purpose-built operating system living permanently inside a device. It performs only the tasks that device needs — nothing more. Found in microwaves, washing machines, ATMs, and cameras, it prioritises small size, low power use, and reliability over flexibility.

1.8.3 Network Operating System (NOS)

Story Mode: The School Computer Lab

Your school has 30 computers. Every student can log in to any machine and access the same files, print to the same printer, and connect to the school's shared internet connection.

How? All 30 computers are connected to a central **server** — a powerful computer managed by a **Network Operating System** that coordinates everything.

When you log in, the NOS authenticates you. When you save a file, the NOS routes it to the server's central storage. When you print, the NOS manages the print queue. You never think about any of this — but the NOS is orchestrating it silently.

What is a NOS?

A **Network Operating System (NOS)** is an OS designed to manage and support multiple computers connected together through a network, enabling them to share resources.

Key functions:

- Manages user accounts across all connected computers
- Allows shared access to files, printers, and internet connections
- Handles network security — who can access what
- Routes data between computers on the network
- Manages a centralised server that stores shared resources

Examples of NOS:

- **Windows Server** (Microsoft's server-focused OS)
- **Linux Server** (used in most web servers worldwide)
- **Novell NetWare** (historically popular in schools and offices)

Where NOS is used:

Environment	How NOS Helps
School computer lab	Students log in to any PC, access personal files from a central server, share one printer
Office	Employees share documents, printers, and databases; centralised IT management
Hospital	Patient records stored centrally; accessible by authorised staff from any terminal
Data centre	Dozens of servers work together, managed by a NOS to distribute computing tasks

NOS vs. General-Purpose OS

Feature	General-Purpose OS	Network OS
Primary focus	One user, one machine	Multiple users, multiple machines
Resource sharing	Limited	Core feature — sharing files, printers, internet
User management	Local accounts only	Centralised accounts across all networked machines
Security	Personal	Network-wide policies and access controls

Pause & Think — 1.8.3

Your school's NOS server crashes on exam day. Thirty students are logged in and taking a computer-based exam.

What happens to the students' ongoing work?

Why is a NOS crash more disruptive than a single student's computer crashing?

What design decisions might a NOS include to recover quickly from crashes?

Quick Recap

A **Network OS** manages multiple computers connected in a network, enabling them to share resources like files, printers, and internet access. Used in schools, offices, and data centres, it provides centralised user management and resource sharing that individual machines could not achieve alone.

1.8.4 Mobile Operating System

Story Mode: From Symbian to Smartphones

In the early 2000s, Nokia's mobile phones ran an operating system called **Symbian**. It could handle basic apps, simple web browsing, and a few games — impressive at the time for a pocket device.

Then in 2007, Apple launched the iPhone with iOS. In 2008, Google launched the first Android phone. These mobile operating systems were built from the ground up for touchscreens, cameras, sensors, wireless connectivity, and a global ecosystem of downloadable apps.

Today, iOS and Android together power over **3 billion smartphones** worldwide. The mobile OS has become the most widely used type of operating system on the planet.

What is a Mobile OS?

A **Mobile Operating System** is an OS designed specifically for smartphones, tablets, smartwatches, and other handheld devices.

Key design priorities:

- **Touch interface** — designed for fingers, not mouse and keyboard
 - **Battery efficiency** — maximise performance while using minimal power
 - **Wireless connectivity** — built-in support for Wi-Fi, Bluetooth, cellular networks
 - **Sensor integration** — camera, GPS, accelerometer, gyroscope, fingerprint reader
 - **App ecosystem** — access to millions of downloadable apps through a centralised app store
 - **Security** — sandboxed apps (each app runs in isolation, cannot access other apps' data without permission)
 - **Always-on awareness** — notifications, background sync, location services
-

Major Mobile Operating Systems

OS	Creator	Used On	Market Share (approx.)
Android	Google (based on Linux)	Samsung, Xiaomi, OnePlus, most brands	~72%
iOS	Apple	iPhone, iPad	~27%
HarmonyOS	Huawei	Huawei phones (primarily in China)	~1%

Mobile OS vs. Desktop OS

Feature	Mobile OS (Android/iOS)	Desktop OS (Windows/macOS)
Input method	Touchscreen, voice	Mouse, keyboard
Power management	Aggressive battery saving	Always plugged in or larger battery
App installation	Curated app store	Download from any website
App isolation	Sandbox (apps isolated from each other)	Less strict isolation
Screen size	4–13 inches	12–34 inches
Processing power	Efficient ARM processors	High-performance x86 processors

Practical Walkthrough: Explore Your Mobile OS

On Android:

1. Go to **Settings**.
2. Scroll down to **About phone** or **About device**.
3. Find: Android version, security patch level, build number.
4. Go back to Settings → **Apps** to see all installed apps and their individual permissions (location, camera, microphone access).

On iOS:

1. Open **Settings**.
2. Tap **General** → **About**.
3. Note: iOS version, model, available storage.
4. Return to Settings → scroll down to any app to see and control its permissions.

Pause & Think — 1.8.4

A food delivery app on your phone requests access to:

- Your **location** (to know your delivery address)
- Your **camera** (to let you scan QR codes)
- Your **microphone** (it wants this, but it's a food delivery app — why?)
- Your **contacts** (it says it helps you invite friends)

Which of these permissions are genuinely necessary for a food delivery app?

Which permissions seem suspicious?

Why does the mobile OS give you the power to grant or deny individual permissions rather than giving every app access to everything?

Quick Recap

A **Mobile OS** is built for touchscreen handheld devices, prioritising battery life, wireless connectivity, sensor integration, and a curated app ecosystem. Android (by Google) and iOS (by Apple) together power the vast majority of the world's smartphones. Mobile OS security uses sandboxing to keep each app isolated from others.

Chapter Summary

Congratulations — you have journeyed through the entire architecture of an operating system. Let us look back at everything you have covered:

Section	Key Idea
1.1 Introduction to OS	The OS is the central controller that translates user actions to hardware instructions and manages multi-user environments
1.2 OS Architecture	The OS is layered: Kernel (engine), Shell (dashboard), Libraries (toolbox), Drivers (translators)
1.3 Process Management	Processes move through lifecycle states; the OS schedules which process runs when using methods like FCFS
1.4 Memory	RAM is fast, temporary primary memory; Virtual Memory extends RAM using slower disk storage
1.5 Processes vs Threads	A process is an independent program; a thread is a task within a process, sharing its memory
1.5.1–1.5.2 Multithreading	Multiple threads run within one process simultaneously, improving speed and responsiveness
1.6 System Calls	System calls are the only legal gateway for programs to request hardware actions from the OS kernel
1.7 File System	Files store data; folders organise them; metadata describes them; file systems (NTFS, EXT4) manage how they are stored
1.8 Types of OS	RTOS (deadlines), EOS (embedded devices), NOS (networks), Mobile OS (smartphones)

Review Questions

Section A: Recall (Answer in one or two sentences)

1. Define an operating system and give one example.
2. What is the difference between the kernel and the shell?
3. What is a process lifecycle? Name the three main stages.
4. Define RAM and explain why it is called "primary memory."
5. What is a system call? Why do programs need it?
6. What is metadata? Give three examples of metadata for a photo.
7. Name the four types of operating systems discussed in this chapter.

Section B: Understanding (Answer in 4–6 sentences)

1. Explain, using a real-world analogy, how the OS layers work together.
2. Describe what happens during FCFS scheduling when three processes arrive at different times. Include a timeline diagram.
3. Explain the difference between multitasking and concurrency. Use an example to illustrate both.
4. Why is virtual memory slower than physical RAM? Use the desk-and-backpack analogy in your explanation.
5. Compare processes and threads. Under what circumstances would you prefer to use threads instead of creating a new process?

Section C: Application and Analysis

1. You are the OS. Three processes arrive: P1 (needs 4s, arrives at 0s), P2 (needs 2s, arrives at 1s), P3 (needs 6s, arrives at 3s). Draw the FCFS timeline, calculate each process's waiting time, and identify which process waited the longest.
2. A hospital is building a new patient monitoring system that tracks heartbeat, blood pressure, and oxygen levels, sending alerts if any reading is critical. Which type of OS would you recommend? Justify your answer with specific reasons.
3. Your friend's computer is running very slowly. You open Task Manager and see that RAM usage is at 98% and disk activity is extremely high. Explain what is happening in terms of virtual memory, and suggest two solutions.
4. A photo editing app crashes while you are editing an important image. Using your knowledge of process management, explain: What state was the process in before it crashed? What happens to the RAM it was using? Will other apps on your computer be affected? Why or why not?

Glossary of Key Terms

Term	Definition
Operating System (OS)	Software that manages hardware, runs programs, and provides a user interface
Kernel	The core of the OS that directly controls hardware resources
Shell	The outer interface of the OS that receives user commands
Process	A running program with its own memory space and system resources
Thread	The smallest unit of execution inside a process, sharing the process's memory
Multitasking	The OS allowing multiple programs to be open and usable simultaneously
Concurrency	Multiple processes being active (in-progress) at the same time
FCFS	First-Come, First-Served — a scheduling method that processes tasks in arrival order
RAM	Random Access Memory — fast, temporary, primary working memory
Virtual Memory	Using storage drive space as extra RAM when physical RAM is full
Multithreading	Running multiple threads within one process simultaneously
System Call	A formal request from a program to the OS to perform a hardware-level action
File System	The method the OS uses to organise, store, and retrieve files on a storage device
Metadata	Data that describes a file (name, size, date, permissions) without being its content
Device Driver	A program that allows the OS to communicate with a specific hardware device
System Library	Pre-written code that programs can use to perform common tasks
RTOS	Real-Time OS — processes tasks within strict, guaranteed time deadlines
EOS	Embedded OS — a minimal OS built into a specific device for a fixed purpose
NOS	Network OS — manages multiple networked computers sharing resources

Term	Definition
Mobile OS	OS designed for touchscreen handheld devices (Android, iOS)
Scheduling	The OS method for deciding which process gets CPU time next and for how long
Paging / Swapping	Moving process data between RAM and virtual memory as needed
Fork	A system call that creates a new process by duplicating an existing one
FAT32 / NTFS / EXT4 / APFS	Different file systems used by different operating systems and devices

End of Unit 1: Operating Systems — Structure and Services

Final Word to the Student: Every time you swipe your phone, open a tab, save a file, or hear music play while you download something else — you are witnessing a symphony of kernel operations, thread scheduling, system calls, and memory management happening in real time. You understand all of that now. You are no longer just a user of these systems. You are beginning to think like the people who build them.